



SAPIENZA
UNIVERSITÀ DI ROMA

Evaluation of Statistical Methods for Generative Password-guessing models

Facoltà di Ingegneria dell'informazione, informatica e statistica
Informatica

Roberto Di Rosa

ID number 2043388

Advisor

Prof. De Gaspari

Co-Advisor

Academic Year 2025/2026

Evaluation of Statistical Methods for Generative Password-guessing models
Sapienza University of Rome

© 2025 Roberto Di Rosa. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: dirosa.2043388@studenti.uniroma1.it

Abstract

TODO

Contents

1	Introduction	1
1.1	Contributions	2
2	Motivation	3
3	Background	5
3.1	Guessability and Password Models	5
3.2	Probabilistic Password Models	6
3.2.1	Explicit Models	6
3.2.2	Implicit Models	6
3.3	Monte Carlo Estimation	6
3.3.1	Rank Definition and Computational Challenges	7
3.3.2	Monte Carlo Rank Estimation	7
3.3.3	Efficient Computation of Monte Carlo Estimates	8
3.3.4	Variance Considerations and Long-Tail Behavior	8
3.3.5	Role of Monte Carlo in Password-Guessing Evaluation	8
3.3.6	Why validation requires deterministic top-k	8
3.4	Markov Chains	8
3.5	Previous Works	10
3.5.1	Fast Accurate And Lean(FLA)	10
3.5.2	Stanek	10
3.5.3	Password Guessing With LLMs	10
3.5.4	MAYA	10
4	Methodology	11
4.1	Optimizing FLA sampling process	11
4.1.1	Pure Python	11
4.2	Cython	12
4.2.1	Cython functions	12
4.2.2	Compilation and Distribution	12
4.2.3	Heap-item	13
4.2.4	Custom-heap	13
4.3	Monte Carlo Implementation	14
4.3.1	IID sampling problems and solution	14
5	Experimental Setup	15
5.1	Dataset	15
5.2	Model and Training	15
5.3	Evaluation Metrics	15
5.3.1	Password Rank Estimation	15
5.3.2	Estimation with Monte Carlo	16

5.4	Implementation Details	17
5.4.1	Generating the Samples	17
5.4.2	Building the Curve	17
5.4.3	Testing and Analysis	17
5.4.4	The <code>heapcy</code> Optimization	18
6	Results and Analysis	19
6.1	Optimizing FLA	19
6.2	Evaluating Monte Carlo	20
7	Conclusion	22
8	Appendix	23
.1	UML	23
.2	Extended FOL	23
.3	Recurrent Neural Networks(RNN)	24
.4	Long Short Term Memory(LSTM)	24
.5	Brute Force	25
.6	Acknowledgements	25
	Bibliography	26

Chapter 1

Introduction

Passwords remain, and will likely continue to remain in the near future, the primary mechanism of human-to-computer authentication [?]. Despite repeated efforts to promote safer practices, users tend to choose weak and highly predictable passwords [?], often following recurring patterns or personal conventions. As a result, the security community has long sought reliable methods to quantify password strength and distinguish weak passwords from strong ones.

A widely adopted metric in this area is *guessability*, which measures how resistant a password is to probabilistic guessing attacks. A password with high guessability requires many attempts before an attacker is expected to generate it, while a password with low guessability can be discovered quickly. Evaluating guessability effectively requires studying passwords from the attacker’s point of view, and this has motivated the development of generative password-guessing models capable of learning complex human password patterns.

Several such models have been introduced in recent years, including FLA [5], PassGAN [3], PassGPT [8], PLRGAN [7], and PassLLM [11]. Frameworks such as MAYA [1] have standardized the evaluation of these models, enabling consistent benchmarking and comparison.

The standard evaluation procedure used in MAYA and in much of the literature relies on generating a *top-k* list of the most likely passwords produced by a model and comparing this list against a test dataset.¹ While conceptually simple, this approach presents a major computational challenge: generating and maintaining the *top-k* list requires processing extremely large numbers of candidate passwords. As observed in both MAYA [1] and FLA [5], the memory footprint associated with storing Python objects (strings, floats, and tuples) grows rapidly and often renders high-quality evaluations intractable without high-performance computing resources.

As an alternative, Monte Carlo estimation [2] offers a way to approximate guessability without exhaustively generating the *top-k* list. Although Monte Carlo methods are effective in practice, their accuracy—particularly for low-probability passwords—depends critically on the size of the sample. Prior works have validated Monte Carlo up to sample sizes of 10^7 , but evaluating it at significantly larger scales has historically been infeasible due to memory limitations.

In the current MAYA implementation, the bottleneck lies in the FLA `get_topk` routine, whose reliance on Python objects introduces substantial overhead. Attempting to use this routine for samples on the order of $N = 5.0 \cdot 10^8$ requires more than 200 GB of RAM, making such experiments impractical on standard hardware.

¹Not all models support explicit probability enumeration; for example, GAN-based models do not inherently generate a *top-k* list.

This thesis addresses this scalability limitation by replacing the Python-based heap structure with a custom Cython implementation. The new design eliminates Python’s object overhead and reduces memory consumption by approximately 87.47% compared to the baseline. This optimization enables evaluations with sample sizes exceeding $N = 5.0 \cdot 10^8$ on commodity hardware, thereby allowing empirical validation of Monte Carlo estimation at scales previously unattainable.

1.1 Contributions

Through these optimizations, we were able to verify the stability of Monte Carlo estimation up to $5 \cdot 10^8$ samples while using only 30 GB of RAM. The principal contributions of this thesis are as follows:

- **A Cython-based optimized heap-item module** (`heap-item`) that reduces memory usage significantly while preserving full compatibility with Python’s `heapq` interface.
- **A custom heap implementation** (`heapcy`) that stores only file offsets rather than complete strings, drastically reducing memory consumption and enabling scalable sampling.
- **Integration of the optimized sampling mechanism into MAYA’s FLA implementation**, allowing more memory-efficient generation of top- k samples.
- **A large-scale Monte Carlo study** demonstrating that rank-estimation accuracy remains stable up to $5 \cdot 10^8$ samples—far exceeding the 10^7 range explored in previous work.

The remainder of this thesis is structured as follows. Chapter 2 discusses the motivation behind this work and the challenges associated with evaluating password-guessing models. Chapter 3 provides background on Monte Carlo estimation, password-guessing models, and related literature. Chapter 4 presents the methodology and describes the proposed optimizations in detail. Chapter 5 outlines the experimental setup. Chapter 6 reports and analyzes the results. Chapter 7 concludes the thesis and highlights potential directions for future research.

Chapter 2

Motivation

Human-chosen passwords remain one of the most widely used authentication mechanisms, yet they continue to exhibit significant weaknesses. Large-scale password leaks occur regularly, providing attackers with extensive datasets from which probabilistic models can be trained. These models learn the statistical patterns of human password choices and are increasingly effective at reproducing them. Understanding the resistance of user-chosen passwords to such guessing attacks is therefore essential both for security research and for practical password-strength estimation systems.

Modern probabilistic password-guessing models assign a probability to each possible password and conceptually enumerate passwords in decreasing likelihood order. A password’s *guess number* represents the number of guesses an attacker, using such an ordering, is expected to make before encountering it. Computing this value exactly is infeasible for models with exponentially large output spaces, since it would require storing or generating all possible strings in sorted probability order.

Monte Carlo estimation offers a practical approximation to this problem. Instead of fully enumerating the model’s distribution, one draws a sample from the model and uses importance-weighted estimation techniques [2] to approximate guess numbers. While Monte Carlo is unbiased and effective in theory, its accuracy—especially for low-probability (long-tail) passwords—depends strongly on the size of the sample. Prior work has shown that variance decreases with larger samples, but validating Monte Carlo at large scales has historically been limited by computational constraints rather than by the method itself.

A practical difficulty arises from the sampling mechanism required to build Monte Carlo estimates. Each sampled password must be stored together with its probability, sorted, and processed to construct the estimation curve. In frameworks such as MAYA, this procedure is implemented using Python data structures, which incur substantial object overhead and memory fragmentation. As a result, sampling large sets—on the order of hundreds of millions of items—quickly becomes infeasible on standard hardware.

The Fast, Lean and Accurate (FLA) model is designed for efficiency and is lightweight enough for client-side deployment. However, when evaluated within MAYA, its Python-based sampling pipeline suffers from the same memory limitations that affect other explicit probabilistic models. The overhead associated with storing millions of Python strings, floats, and tuples restricts the maximum feasible sample size, thereby limiting the ability to study Monte Carlo estimation at the scales needed to assess its stability for password-strength evaluation.

The motivation behind this thesis is therefore twofold. First, we aim to remove the memory bottlenecks inherent in Python-based sampling by designing more

compact, low-level data structures that can be used transparently within MAYA. Second, by enabling substantially larger sample sizes, we seek to empirically assess the accuracy and practicality of Monte Carlo estimation for password-guessing models at a scale that has not been previously attainable. Through these optimizations, we enable faster, more memory-efficient sampling and create the conditions necessary to evaluate Monte Carlo methods reliably for password-strength modeling.

Chapter 3

Background

3.1 Guessability and Password Models

P.P.Ms are a necessary part of a modern login security system they are needed to evaluate the strength of the user's password, since today's attackers also use these models to crack passwords more efficiently. When defending they evaluate the strength of a given password by telling you how many attempts it would take to guess it this is known as the **Rank** of a password, the higher the rank the more secure the password. Attackers prefer to use these models instead of brute forcing. We could use optimized versions like [John The Ripper](#), which speed up the brute forcing job but are still very slow compared to these models. These probabilistic models instead rely on something else, after training we take all of the tuples of

1. password
2. password's probability

and put them inside of a set Γ ,¹ we can now find out the rank of a given password α .

$$R(\alpha) = |\{\beta \mid \beta, p_\beta \in \Gamma \wedge p_\beta > p(\alpha)\}| \quad (3.1)$$

This will give us the rank of a password, or more simply, the amount of attempts it would take to guess it, the main problem from this approach is its data intensity the set of strings can be really big. We measure the efficacy of the model by seeing in how many attempts it takes to guess its test-dataset completely, the formula to do this is very simple:

$$G(x) = \frac{|\mathcal{S}(x) \cap \mathcal{T}|}{|\mathcal{T}|} \quad (3.2)$$

$x \in \mathbb{N}$ is the number of guesses the model makes, $\mathcal{T}$ is the set of all the test-dataset passwords, $\mathcal{S}(x)$ is the top-x sampling function that gives us the top-x most probable strings to be generated by the model. $G(x)$ is the ratio of the intersection between the sample set and the test-dataset,

They work by getting trained on a dataset(e.g. RockYou,Libero,TaoBao,etc...), from these datasets they will take the most frequent passwords and they will create distributions for these passwords in a way as to reflect the frequency of characters

¹If the model we are working with is implicit, we don't have access to any distribution, therefore at first it's impossible to use calculate the rank of a password in these models, without any modification/new model.

all of this using one of these RNNs, Markov chains, PCFGs, LSTMS, Random Forests[10], They can be separated into 2 types:

1. **Trawling** are aimed guessing correctly as many passwords as possible[11] and are constrained by the amount of resources and the number of guesses itself.
2. **Targeted** are aimed at guessing the password of a single user given some specifics (e.g.: Name,Surname,Sex,Nationality,etc...)

3.2 Probabilistic Password Models

3.2.1 Explicit Models

- FLA [5] (Fast, Lean, and Accurate), was the first approach to apply neural networks to the password-guessing task. It is the only approach we selected based on a recurrent neural network, specifically an LSTM, allowing us to examine how models designed for sequential data processing perform compared to other, more recent generative architectures [1].
- Markov Chains
- PCFGs

3.2.2 Implicit Models

- **PassGAN** [3] is based on a Generative Adversarial Network (GAN) architecture, which, unlike other designs, follows an adversarial training approach. As they are implicit models, GANs learn to generate data by capturing the underlying distribution of the training set without explicitly defining a probability distribution, offering a unique perspective in password generation [1].
- **PLRGAN** is an enhanced version of PassGAN and represents the current state-of-the-art for GAN-based methods. PLR-GAN further adopts a Dynamic Password Guessing (DPG) strategy, which allows the model to adapt its guesses based on the distribution of successfully guessed passwords, increasing the likelihood of generating relevant guesses [1].
- PassGPT proposes a GPT-2-based language model for password generation. Similarly to FLA, PassGPT employs a sequential generation process. However, GPT-2 is built upon an attention mechanism, which allows it to capture long-range dependencies more effectively. [1]

3.3 Monte Carlo Estimation

Given an analytically complex problem to solve, the use of Monte Carlo estimation provides a value close to the true solution, despite being analytically simpler. If a sample of n items is drawn from the main problem, we then perform a sum over the values, then divide at the end by $\#samples$. This will give us an approximation of the expected value of our original problem. Let us say that we do not know the

probability of a coin toss, we can use Monte Carlo to estimate this probability:

Given a Bernoulli random variable $X \in \{0, 1\}$

$$\hat{p} = \frac{1}{n} \sum_{i=0}^{n-1} X_i$$

As shown in the formula, above and according to the weak law of large numbers ²

3.3.1 Rank Definition and Computational Challenges

The Rank of a password α is defined as such:

$$R(\alpha) = |\{\beta \mid \beta, p_\beta \in \Gamma \wedge p_\beta > p(\alpha)\}| \quad (3.3)$$

The Rank of a password α is defined as such:

$$R_f(\alpha) = |\{\beta \mid \beta, p_\beta \in \Gamma \wedge p_\beta > p(\alpha) \wedge f(\beta)\}| \quad (3.4)$$

This shows us that we can filter during sampling as long as we normalize during the sampling not after.

3.3.2 Monte Carlo Rank Estimation

Monte Carlo's estimation is usefull in this field [2] because it enables us the confidence of estimating the rank of a password, and by extension how good a model is how does it work? we start by defining a set Θ , it will contain a sample of strings taken from a i.i.d.³ distribution Ψ based on the internal one of the model. Furthermore the larger the size of Θ the better the estimation as proved in [2], this will help us especially with Low-probability passwords since they have higher variance. In this paper, we implemented the version found in [2] without any improvements as seen in [9] this has not impacted on the performance of the guess% estimation. To this day we have not found a way to better this variance, the latest attempts have yielded some results in lowering the variance therefore making montecarlo more accurate especially for Low-probability passwords, but they are not good enough yet. We can now Estimate the **Rank**, we define $\hat{R}(\alpha)$ as the estimation of the Rank for a given password α :

$$\hat{R}_\Psi(\alpha) = \hat{\mathbb{E}}_{\beta \sim \Psi} \left[\frac{\mathbb{1}[p(\beta) > p(\alpha)]}{p(\beta)} \right] = \frac{1}{n} \sum_{\beta \in \Theta} \frac{\mathbb{1}[p(\beta) > p(\alpha)]}{p(\beta)} [2] \quad (3.5)$$

$$\hat{R}_\Psi(\alpha) = \hat{\mathbb{E}}_{\beta \sim \Psi} \left[\frac{\mathbb{1}[p(\beta) > p(\alpha) \wedge f(\beta)]}{p(\beta)} \right] = \frac{1}{n} \sum_{\beta \in \Theta} \frac{\mathbb{1}[p(\beta) > p(\alpha)]}{p(\beta)} [2] \quad (3.6)$$

²The weak law of large numbers states that an observed sample average from a sample will be closer to the true population average as the sample grows larger. [4]

³Independently, Identically Distributed we need this because if the distribution is not i.i.d. therefore we will not be able to get accurate estimates

3.3.3 Efficient Computation of Monte Carlo Estimates

This is a good estimation but if we need to estimate a lot of passwords, this will not be fast since each time we need to sample for montecarlo and calculate the rank, so the solution is to precalculate the rank and samples so that we do not need to do it multiple times, so let $\mathbf{A}$ be the descending sorted array of probabilities for getting generated by the model for each sampled password, and let $\mathbf{C}$ be the array of the estimated ranks for each password.

$$\text{Let } \mathbf{A} = [p(\beta_0), p(\beta_1) \dots, p(\beta_n - 1)]$$

$$\text{Let } \mathbf{C}[i] = \frac{1}{n} \sum_{j=0}^i -1 \frac{1}{A[j]} = C[i - 1] + \frac{1}{n \cdot A[i]}$$

So given a password α to find its rank given $\mathbf{A}$ and $\mathbf{C}$, we need to reverse binary search in $\mathbf{A}$ so that we find the index i of the probability which is lower than $p(\alpha)$, We than take the value stored at $\mathbf{C}[i]$ which will be the $\hat{R}(\alpha)$, so we found our solution really fast.

3.3.4 Variance Considerations and Long-Tail Behavior

Using Monte Carlo estimation for high probability passwords is not a problem but as soon as we start estimating the rank of low probabilities, the variance explodes.

3.3.5 Role of Monte Carlo in Password-Guessing Evaluation

Let G be the set of guess numbers from the Monte Carlo estimation and $N = |G|$ be the sample size. The crack rate for a guess budget x is defined as:

$$R_{\text{crack}}(x) = \frac{|\{g \in G \mid g < x\}|}{N} \quad (3.7)$$

3.3.6 Why validation requires deterministic top-k

Validation requires deterministic top-k sampling because that is the ground truth.

3.4 Markov Chains

Markov chains provide a framework through which we can model sequential events. It works as follows: we have a set of states for each state we have associated a probability distribution of going to another, that can be easily represented with a matrix. To formalize this we can look at the below representation

$$\text{Let } S = \{S_1, S_2, \dots, S_n\} \text{ be the state space.}$$

$$\text{Let } P \text{ be the } n \times n \text{ transition matrix, where}$$

$$P_{ij} = P(X_{t+1} = S_j \mid X_t = S_i)$$

We can see applications of this in fields such as: NLP⁴, PPM⁵ etc... Markov Chains are often the basis for prediction models such as Probabilistic Password Models, but they are not enough to model these problems in fact in recent years we started using different kinds of Password-guessing models, like LLM based ones .

⁴Natural Language Processing

⁵Probabilistic Password Models

3.5 Previous Works

3.5.1 Fast Accurate And Lean(FLA)

FLA is an LSTM model that is used to guess passwords and study how effective an attacker might be at cracking passwords. As evidenced in [5] the model is very good at guessing passwords compared to PCFGs, Markov Models and more heuristic methods like John The ripper. Although in recent years there has been a development in to password guessing, as in the emergence of new kinds of model like: RFGuess [10], PASS LLM [11] etc... The authors of FLA have done an excellent job at making it very "lean" so that it can run easily on the client side, making it a really good model for password strength metering on the client-side which was a novel idea at the time. They also used Monte Carlo estimation[2] to efficiently look at how many guesses it would take to guess all of the test dataset [5], and proved reliability up to 10^7 .

3.5.2 Stanek

In Stanek's paper [9] they improved the Monte Carlo Estimation of the Rank of Passwords by employing different methods: they first addressed the precision problem with two ideas:

1. **Linear Interpolation** We take the passwords in rank intervals and apply linear Interpolation⁶, this will give us results with lower variance.
2. **Uniqueness post-sampling** After sampling we sample until there are no repeated probabilities, and substitute them with a frequency.

They then proceeded to address the speed problem, with the idea of separating the probabilities in buckets

3.5.3 Password Guessing With LLMs

As seen in [11] they brought forward the concept of using LLMs for password guessing and obtained really good results in the evaluation of guesses. These LLMs can get to higher guessing numbers much faster than previous implementations, instead of previous models.

3.5.4 MAYA

MAYA's framework introduces a standardized and extensible framework to test models

⁶Linear Interpolation is a common technique used to lower the variance of a Monte Carlo Estimation <https://www.math.hkust.edu.hk/~machas/scientific-computing.pdf>

Chapter 4

Methodology

4.1 Optimizing FLA sampling process

In this section, we start by analyzing the problems we encountered with MAYA and discussing the solutions we found to the said problems, their strengths, and weaknesses.

The problem we encountered was the need to sample the top-k elements from the distribution within FLA. To address these limitations, we explore two distinct strategies: The first, *Heap-item*, focuses on reducing item size via Cython structs. The second, *Custom-heap*, introduces a file-offset mechanism to eliminate string storage entirely

4.1.1 Pure Python

The first problem we tackled was the original implementation of FLA’s sampling algorithm[5], which occupied a significant memory usage due to the usage of pure Python structures such as lists, tuples, strings, and floats. The main problem came from the fact that each item was saved as a tuple of str, float, which was appended into a list, and that list was manipulated with the `heapq` module. Figure 1 illustrates the significant memory consumption caused by the original technique. Python heap structures store each heap item as a fully-fledged Python object (tuple of string and float). This leads to high memory overhead due to:

1. object headers,
2. reference counting,
3. heap fragmentation,
4. garbage collector metadata,
5. repeated storage of identical strings.

As a result, storing 10^6 heap entries may require hundreds of megabytes, when combined with all of the other objects we keep in memory this results in excessive memory usage, on the order of 200 GB when sampling $5 \cdot 10^8$ making it impossible to scale Θ to the sizes required for big model Evaluation.

4.2 Cython

Cython (not to be confused with CPython) is a superset of Python. It is a compiled language, rather than interpreted, and allows for the creation of C/C++ extensions for Python with ease. The typical workflow is as follows: We first define compiler directives (starting with `#`). These tell the compiler how to translate the code to C/C++ and handle bindings. We then write the logic using Python-like syntax.

4.2.1 Cython functions

Cython provides three types of function declarators:

- **cdef**: Defines a C-only function. These are very fast but cannot be called from external Python code. By default, they act like standard Python functions and hold the [GIL](#)¹, allowing access to Python objects. However, if defined with `nogil` (e.g., `cdef void func() nogil:`), they release the GIL; in this mode, Python structures cannot be accessed.
- **cpdef**: Creates two versions of the function: a C version for use within Cython and a Python wrapper for external use. This offers the best of both worlds but introduces slight overhead compared to pure `cdef`.
- **def**: These are standard Python functions. They are accessible from anywhere but incur the standard Python function call overhead.

This distinction allows us to write performance-critical loops/operations in `cdef` functions while keeping the interface accessible via `cpdef` or `def`.

Caller	Can call cdef?	Can call cpdef?	Can call def?
Cython (cdef/cpdef)	Yes	Yes	Yes
External Python	No	Yes	Yes

Table 4.1. interaction matrix between Python and Cython execution contexts. `cdef` functions optimize performance by bypassing the Python interpreter but are inaccessible to external Python modules

4.2.2 Compilation and Distribution

Cython's compiler will output `.whl` files that then can be imported into Python files like normal. The problem is that it can be burdensome to compile for every Python Version Header with only the Cython compiler. Therefore we opted to use [cibuildwheel](#) which when integrated with Github's CI/CD this facilitates software distribution.

¹The Global Interpreter Lock is a Python feature which was introduced to eliminate race conditions in parallel operations on objects, Python needs it because objects are tracked by the Garbage collector by keeping count of the references of said object when the references reach 0 then the garbage collector frees memory, if there are parallel operations on the same object this could lead to an early free inducing a use after free, or race conditions, we resolve this by using the GIL which makes each operation on objects one thread only thus preventing the majority of problems from arising.

4.2.3 Heap-item

The first proposed solution was to build a [heap-item](#) custom module written in Cython thus maximizing performance and compatibility with Python. A significant limitation of this approach is that, similar to the pure Python implementation, each heap item remains a distinct object managed by the garbage collector., each heap item is an object that must be tracked by python's [garbage collector](#), nevertheless this approach helped us reduce memory consumption by $\sim 59.79\%$ instead of the pure python approach as seen in Figure 1, however further optimization was necessary, so we opted for a custom heap called [heapcy](#).

Using Cython allows us to replace Python objects with C-structured objects giving us improved memory usage while also giving us ease of integration in Python, we could have used C++ and then have written the integrations with [nanobind](#), but that would have taken considerably more time and the fast software distribution would have been more complicated. Since we use Cython we have access to [cibuildwheel](#) which when integrated with github CI/CD allows us to publish binaries to PyPi rendering software distribution really easy. These Cython objects:

- are allocated in contiguous memory,
- avoid Python reference counting,
- do not trigger garbage collection,
- store fields compactly (the struct is made using chars and doubles which is much smaller than normal python strings, and floats)

This reduces per-item memory usage tremendously. However, each item is still an Python object, so total memory still scales linearly with the number of stored items.

4.2.4 Custom-heap

Rather than simply implementing a heap in Cython, we adopted a different approach by storing only the string's position in a file as a reference. This method significantly reduced memory usage, as shown in Figure 1, and required only a single heap object instead of multiple instances as in previous implementations [4.1.1](#) and [4.2.3](#). We first copy everything which was guessed by the `complete_guessing` from the zipped output file to a tmpfile created in the `/tmp` directory. The copy is made with the stream-based copying, after copying we keep the tmp-file open and then we start to take the strings position with the `tell()` method, we then read the line and save it all in the heapcy. After the heap we finish saving all of the heap elements, we start taking all of the offsets in decreasing order by probability, into a list, which will be passed to the helper method included into heapcy with generatorm pattern which will occupy $O(1)$ of memory and it will give us the strings one by one in $O(1)$ since we use the `seek()` method, which in binary files is $O(1)$. The I/O overhead added by the reading/writing of the files does not make the program take more time since the Cython optimizations counter said overhead, in our testing we registered for a sampling of 100_000 items: 2 hours and 40 minutes for the unoptimized version and 2 hours and 37 minutes for the optimized one. Therefore we are confident to say that this optimized version is good enough since, it has given us a memory consumption improvement of $\sim 87.47\%$ over [4.1.1](#) and a $\sim 68.85\%$ over [4.2.3](#) While this solution offers superior memory performance, it is more complex to use compared to the

previous solutions, so the choice should depend on the specific requirements of the development environment.

Since we do not actually need strings in memory during the sampling process, we can write a password once to a binary file, and the heap can store only its integer file offset together with the probability value.

This design:

- avoids storing millions of Python strings,
- eliminates repeated allocations,
- stores heap entries as compact C structs,
- ensures that the heap consists of a single contiguous memory region,

The tradeoff is increased implementation complexity and the need for careful file handling, but the performance benefit is decisive.

4.3 Monte Carlo Implementation

4.3.1 IID sampling problems and solution

The sampling used in the MAYA implementation was not IID because it had a thresholds for the low probability passwords, therefore Since Monte Carlo estimation requires a huge number of calculations, doing them one by one would be too slow. Therefore, we implemented a custom batched sampler using PyTorch. This allows us to use the GPU to process thousands of samples at the same time. This approach makes the estimation much faster compared to a standard iterative approach on the CPU.

Chapter 5

Experimental Setup

5.1 Dataset

For this paper we chose the [RockYou](#) dataset the same as in MAYA [1](split 80;20), for time reasons we couldn't test with other datasets. RockYou is the standard dataset used in nearly all password-guessing literature. Using it ensures comparability with previous work and aligns with the datasets used by MAYA and FLA. Its size and structure also provide statistically meaningful evaluation for both trawling and targeted password-guessing attacks.

5.2 Model and Training

As discussed, there was no time to implement the functions required to test with other models. Model wise we used FLA found in MAYA[1], the model was pre-trained on the RockYou training set with a maximum password length of 12 characters, using the `rq1` test configuration. This setup isolates the impact of the memory optimizations, allowing for a direct assessment of the sampling efficiency without confounding variables from differing model architectures.

To evaluate these models we need to see how accurate their guessing [5] is, the following section will show us how to do this in multiple ways.

5.3 Evaluation Metrics

5.3.1 Password Rank Estimation

Limitations of Deterministic Ranking.

While using the deterministic rank would give us the most precise result, it requires an enormous amount of resources since we first need to calculate the entire set of passwords yieldable by the model, so we opt for the estimation As described above, to get our guessing percentage estimate we first need to implement the Monte Carlo seen in [2]. To do this we need 1 set:

$\Theta := \{(pwd, prob) \mid (pwd, prob) \in \text{getTheta}(\text{model}, \text{sampleSize})\}$ our Monte Carlo sample.

5.3.2 Estimation with Monte Carlo

We employ a Monte Carlo estimate (5.1) to get a result similar to (??). $\hat{R}$, represents the estimate of the rank (number of times the model needs to guess¹) to generate the password.

Thanks to this we require much less effort compared to the deterministic way, since all we need to do is to compute the Monte Carlo curve once per Model Checkpoint, save it to .csv file and then for each guess number apply the formula above and we will have our results significantly faster than the previous approach.

The former approach was very slow since we needed to take the top_k where k is the guess number [1].

There are two ways to calculate the Monte Carlo estimation both specified in [2] and here included:

Eq. Monte Carlo Estimation for the rank of a password.

$$\hat{R}_{\Psi}(\alpha) = \hat{\mathbb{E}}_{\beta \sim \Psi} \left[\frac{1[p(\beta) > p(\alpha)]}{p(\beta)} \right] = \frac{1}{n} \sum_{\beta \in \Theta} \frac{1[p(\beta) > p(\alpha)]}{p(\beta)} \quad (5.1)$$

Array sorted in descending order

$$\text{Let } \mathbf{A} = [p(\beta_1), p(\beta_2) \dots, p(\beta_n)] \quad (5.2)$$

Eq. Optimized version

$$\text{Let } \mathbf{C}[i] = \frac{1}{n} \sum_{j=1}^i \frac{1}{A[j]} = C[i-1] + \frac{1}{n \cdot A[i]}$$

Estimation of the guess number for a given x.

$$\hat{G}(x) = \frac{1}{|T|} \sum 1_{[\hat{R}(\alpha) \leq x]} \quad (5.3)$$

Eq. Variant of (5.3)

$$\hat{G}(x) = \frac{|\{\alpha \in \mathcal{T} \mid \hat{R}(\alpha) \leq x\}|}{|\mathcal{T}|} \quad (5.4)$$

¹As defined in [2]

5.4 Implementation Details

The whole project is written in Python. We decided to use PyTorch mainly because we needed to do a lot of matrix math really fast, especially for the Monte Carlo part which gets heavy on the CPU.

5.4.1 Generating the Samples

The process starts with the function `get_theta`. This is basically the setup phase. Inside here, we build the specific model we want to test (like a PCFG or LSTM) using a simple factory logic. This means we don't have to change the code manually every time we switch models.

Once the model is ready, we call `OneBatchToControlThemAll`. As the name suggests, this function controls the main loop. We cannot just generate 100 million samples at once because the GPU memory would crash. So, this function splits the work into small batches (usually 2048 samples) and runs them one by one.

For every batch, the actual work happens in `IIDSampleBatched`.

- We create a batch of empty strings.
- We loop through the password length.
- In every step, we ask the model for the probabilities of the next character for the *whole batch* at once.
- We pick the next characters and update the probabilities.

When a batch is done, we append the results (password and probability) to a CSV file. This file effectively becomes our Θ (the set of samples).

5.4.2 Building the Curve

After we have all the samples in the CSV, we need to turn them into something we can use to rank passwords. We do this in `_build_mc_curve`.

The goal is to get two arrays:

1. **Array A:** The probabilities of all our samples, sorted from highest to lowest.
2. **Array C:** The estimated rank for each probability.

Sorting millions of numbers takes a while. So, we added a check: if we already calculated these arrays before, we just load them from the disk. If not, we read the CSV, sort the probabilities, and calculate the cumulative sum to get the ranks. Then we save them immediately so we don't have to wait next time.

5.4.3 Testing and Analysis

To test real passwords, we use the `MonteCarloTest` function. It's very simple: we take the probability of a password from our test set and use binary search to find where it fits in our sorted Array A. The index tells us the position, and Array C gives us the rank.

Finally, for the plots, we use `ComputeCrackRates`. We define some guessing budgets (like 10^6 or 10^8 guesses) and check what percentage of our test passwords were guessed within that budget. We save these numbers to a TSV file to draw the graphs later.

5.4.4 The heapcy Optimization

Chapter 6

Results and Analysis

6.1 Optimizing FLA

We can see that our optimization work perfectly in the sense that "fiat iustitia et pereat mundus"

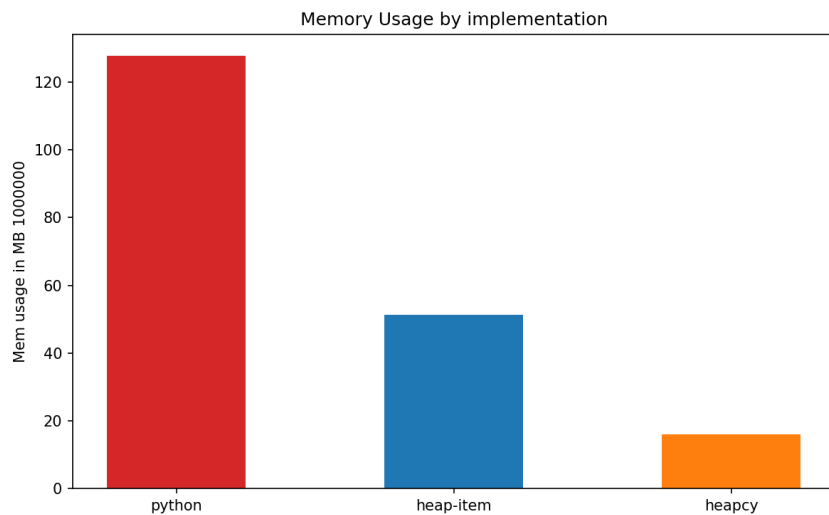


Figure 1. Comparative analysis of memory consumption across the three sampling strategies for a batch of 10^6 items. The **heapcy** (Custom-heap) implementation demonstrates a $\sim 87.47\%$ reduction in memory usage compared to the baseline **Pure Python** approach. This drastic drop is attributed to the file-offset mechanism (Section 4.2.4), which eliminates the overhead of storing string objects in RAM. The reduction in memory is not only an efficiency gain; it changes the problem class from 'requires a supercomputer' to 'runs on a laptop'.

#Item	Pure Python	Heap Item	Heapcy
10^6	127.72	51.36	16.00
10^5	12.77	5.13	1.60

Table 1. Size occupied in MB for 10^x amount of items by each implementation, this tells us how good the optimizations are instead of the old implementation

As can be seen the gains have the same rate at any amount of items, so by using 10x the amount of items we will have 10x the amount of memory used.

6.2 Evaluating Monte Carlo

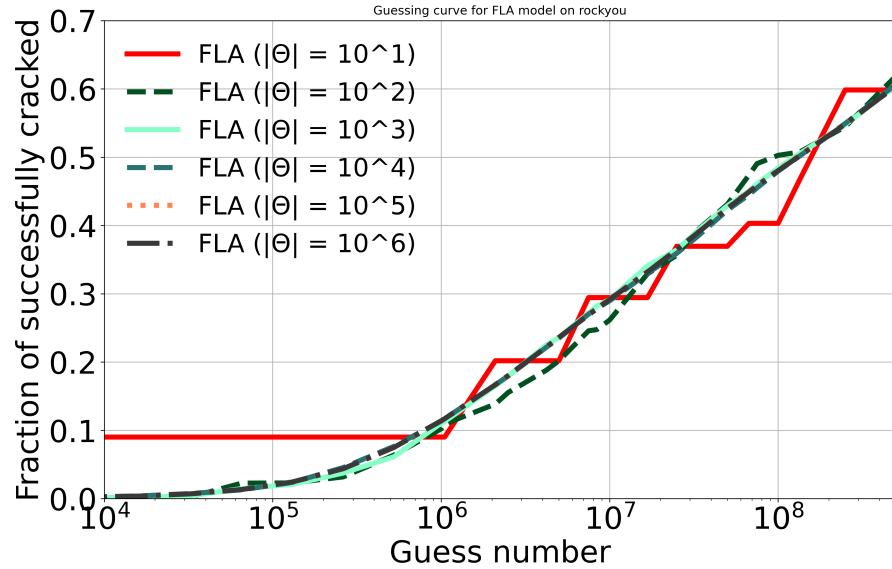
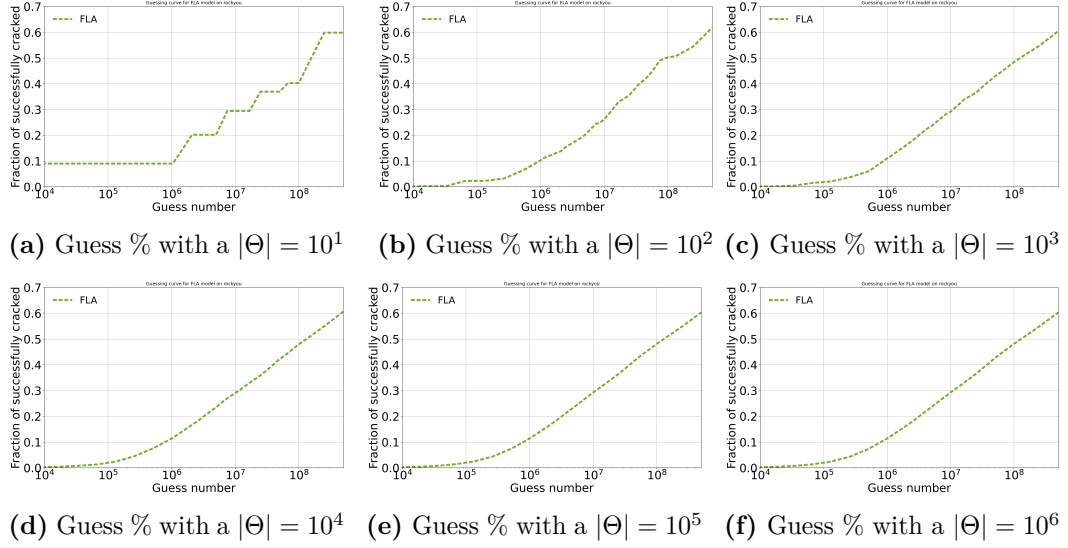


Figure 2. Convergence of the Monte Carlo rank estimation as the sample size $|\theta|$ increases. Subfigures (a) through (d) illustrate that at lower sample sizes ($10^1, 10^2, 10^3$), the estimation exhibits high variance (noise). As $|\theta|$ approaches 10^5 and 10^6 (f), the curve stabilizes confirming that sample sizes large enough produce deterministic like accuracy.

Guess #	Rate 10^1	Rate 10^2	Rate 10^3	Rate 10^4	Rate 10^5	Rate 10^6	MAYA %
$1.0 \cdot 10^6$	9.01%	10.22%	11.03%	11.35%	11.37%	11.40%	11.40%
$2.5 \cdot 10^6$	20.19%	15.54%	17.91%	17.95%	17.97%	18.02%	18.04%
$5.0 \cdot 10^6$	20.19%	20.19%	23.60%	23.49%	23.57%	23.61%	23.66%
$7.5 \cdot 10^6$	29.43%	24.57%	27.17%	27.02%	26.94%	26.90%	26.94%
$1.0 \cdot 10^7$	29.43%	26.15%	29.15%	28.97%	29.25%	29.16%	29.20%
$2.5 \cdot 10^7$	36.95%	35.63%	36.41%	35.91%	36.42%	36.45%	36.48%
$5.0 \cdot 10^7$	36.95%	43.09%	42.88%	42.23%	42.59%	42.47%	42.49%
$7.5 \cdot 10^7$	40.31%	49.05%	46.09%	45.50%	45.84%	45.80%	45.82%
$1.0 \cdot 10^8$	40.31%	50.26%	48.43%	47.95%	48.11%	48.02%	48.04%
$2.5 \cdot 10^8$	59.84%	54.45%	54.74%	54.92%	54.85%	54.83%	54.85%
$5.0 \cdot 10^8$	59.84%	61.87%	60.45%	60.65%	60.35%	60.42%	60.47%

Table 2. Stability of the Guessing Percentage ($G(x)$) across varying Monte Carlo sample sizes. The data indicates that even at low samples 10^4 the results are close enough to the deterministic ones.

Chapter 7

Conclusion

In Conclusion we managed to optimize the sampling of FLA, added montecarlo to MAYA and we proved with results how the montecarlo estimation is very good at estimating the evaluation. We went beyond FLAs original paper by proving the result up to $5.0 \cdot 10^8$ this may imply that we do not need to the evaluation deterministic enumeration, by using enumeration, instead we can use montecarlo to see how good a model really is. Being able to extend Monte Carlo sampling to $|\Theta| = 5 \cdot 10^8$ is important, since the accuracy of it is entirely dependent on the size of $|\Theta|$. The larger the sample size the smaller the variance in the tail of the distribution (Low probability passwords), which is where many real passwords reside. Our optimizations therefore enable a better evaluation of password-guessing models without requiring enormous amounts of memory usage. In future works we should expand this method to every model implemented inside of MAYA, and try montecarlo for even larger numbers, since the new optimization allows us to enumerate up to even bigger numbers without incurring into big performance issues. We should as well optimize the caching part, since for now it is all sequential and takes a considerable amount of time $\sim 4hr$ for RockYou test-dataset $\sim 4.0 \cdot 10^6$ passwords. We could also improve the heapcy implementation to have a dynamic memory instead of the current implementation.

Chapter 8

Appendix

.1 UML

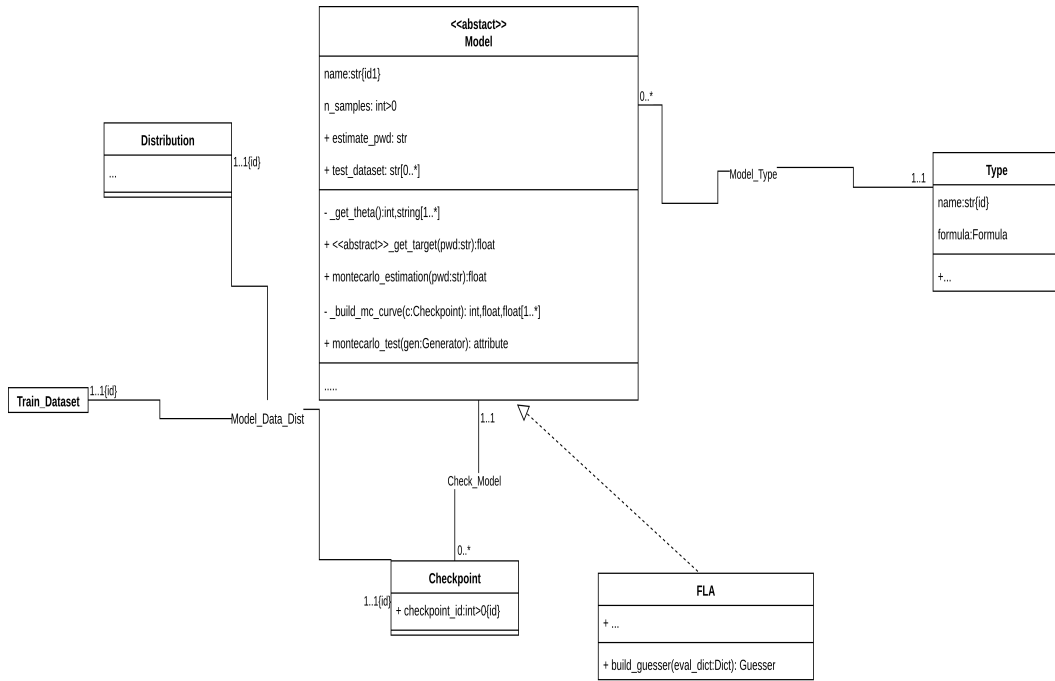


Figure .1. Analysis

.2 Extended FOL

Let $\mathbf{A} := \{(i, prob) \mid i \in \mathbb{N} \wedge (x, prob) \in sorted(\{x, p \mid \exists pwd \in \Theta \wedge x \in \mathbb{N} \wedge P(pwd, p) \wedge x = x + 1\}, sort_func) \wedge i = |\mathbf{A}| + 1\}$ (5.2)

```

sort_func( $t_1 : \text{int} \geq 0, \text{real}(0..1), t_2 : \text{int} \geq 0, \text{real}(0..1)$ )  $\rightarrow \text{Bool} :$ 
  preconditions : None
  postconditions :
    | ret :=  $\forall n_1, n_2,$ 
    |  $\text{Real}(t_1, n_1) \wedge \text{Real}(t_2, n_2) \implies n_1 > n_2$ 
    | return ret

```

$$\text{Let } \mathbf{C} := \{(i, n) \mid \exists x, \text{prob} \wedge i \in \mathbb{N} \wedge n \in \mathbb{R}_{>0} \wedge |A| > 0 \wedge n = \frac{1}{|A|} \cdot \sum_{x, \text{prob} \in A}^{i \leq x} \frac{1}{\text{prob}}\}$$

.3 Recurrent Neural Networks(RNN)

Models that have 3 layers¹:

1. Input Layer X
2. Hidden Layer H
3. Output Layer Y

This is what happens at each step t: the input vector x_t is used by the RNN to update its hidden state h_t , with this function[6]:

$$h_t = \sigma_h(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad (.1)$$

W_{xh} is the matrix that represents the weights between the input layer and the hidden one[6], W_{hh} is the weight matrix for the recurrent connection and b_h is the bias vector.

At the end the model will update its y_t state in this manner [6]:

$$y_t = \sigma_y(W_{hy}h_t + b_y) \quad (.2)$$

W_{hy} is the matrix that represents the weights between the hidden layer and the output layer[6], and b_y is the bias vector. The recurring part is the computation of the hidden state at each step. However, this model type does not have good "Memory" because, during the hidden state calculation, the gradient starts to "vanish." This means that the model cannot learn long sequences of words. To solve this problem, there are many evolutions of this kind of model.

.4 Long Short Term Memory(LSTM)

This is an evolution of RNN's which mitigates the vanishing gradient problem by controlling it. They introduce the concept of these 3 gates:

1. input

¹This is a simplification [6]

2. forget
3. output

Each of these tell the model how much of the previous, input, hidden and output states to consider from the previous state. Even though we have this improvement over the base model, this is still not perfect as the vanishing can still occur, nevertheless it is better than the normal one[6]. LSTMs are particularly effective for password modeling because they capture long-range dependencies better than normal RNNs or simple Markov Chain Models.

.5 Brute Force

Brute forcing is a procedure in which we try every possible password to find the correct one e.g. "What is Roberto Di Rosa's Gmail password?" Let us assume he used only characters from the english alphabet and that the password has a maximum length of 12 characters, so we know that there are 26 characters in the english alphabet let's start by the first:

1. a
2. b
3. c ...
4. aa
5. ab
6. ac

We will try until we reach our result, the problem with this approach is that we might have to try: $26^{12} \sim 95.43$ quadrillions times, which for any pc is very computationally demanding.

.6 Acknowledgements

"Don't be overwise; fling yourself straight into life, without deliberation; don't be afraid - the flood will bear you to the bank and set you safe on your feet again."
Fyodor Dostoevsky

Bibliography

- [1] CORRIAS, W., DE GASPARI, F., HITAJ, D., AND MANCINI, L. V. MAYA: A unified benchmarking framework for generative password-guessing models. In *2026 IEEE Symposium on Security and Privacy (SP)* (2026). ArXiv preprint arXiv:2504.16651v2. Available from: <https://arxiv.org/abs/2504.16651v2>.
- [2] DELL'AMICO, M. AND FILIPPONE, M. Monte carlo strength evaluation: Fast and reliable password checking. In *Proceedings of the 22nd ACM Conference on Computer and Communications Security (CCS'15)*. ACM Press (2015). Available from: <https://dl.acm.org/doi/pdf/10.1145/2810103.2813631>.
- [3] HITAJ, B., GASTI, P., ATENIESE, G., AND PEREZ-CRUZ, F. Passgan: A deep learning approach for password guessing. In *Proceedings of the 17th International Conference on Applied Cryptography and Network Security (ACNS 2019)*, pp. 217–237. Springer (2019). Available from: https://link.springer.com/chapter/10.1007/978-3-030-21568-2_11.
- [4] LOÈVE, M. *Probability Theory I*. Springer, New York, 4th edn. (1977). Available from: https://archive.org/details/probabilitytheor0000loev_h6n0/mode/2up.
- [5] MELICHER, W., UR, B., SEGRETI, S. M., KOMANDURI, S., BAUER, L., CHRISTIN, N., AND CRANOR, L. F. Fast, accurate, and lean: A low-overhead, high-accuracy method for a-priori password strength estimation. In *25th USENIX Security Symposium (USENIX Security 16)* (2016). Available from: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/melicher>.
- [6] MIENYE, I. D., SWART, T. G., AND OBAIDO, G. Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, **15** (2024). Available from: <https://www.mdpi.com/2078-2489/15/9/517>.
- [7] PASQUINI, D., GANGWAL, A., ATENIESE, G., BERNASCHI, M., AND CONTI, M. Improving password guessing via representation learning. In *Proceedings of the 42nd IEEE Symposium on Security and Privacy (SP 2021)*, pp. 1382–1399. IEEE (2021). Available from: <https://ieeexplore.ieee.org/document/9519441>.
- [8] RANDO, J., PEREZ-CRUZ, F., AND HITAJ, B. Passgpt: Password modeling and (guided) generation with large language models. In *Proceedings of the 28th European Symposium on Research in Computer Security (ESORICS 2023)*, pp. 164–183. Springer (2023). Available from: https://link.springer.com/chapter/10.1007/978-3-031-51479-1_9.

- [9] STANEK, M. Revisiting monte carlo strength evaluation. *arXiv preprint arXiv:2408.00124*, (2024). Available from: <https://arxiv.org/abs/2408.00124v1>, [arXiv:2408.00124](https://arxiv.org/abs/2408.00124).
- [10] WANG, D., ZOU, Y., ZHANG, Z., AND XIU, K. Password guessing using random forest. In *32nd USENIX Security Symposium (USENIX Security 23)* (2023). Available from: <https://www.usenix.org/conference/usenixsecurity23/presentation/wang-ding-password-guessing>.
- [11] ZOU, Y., AN, M., AND WANG, D. Password guessing using large language models. In *USENIX Security Symposium (USENIX Security 25)* (2025). Available from: <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-yunkai>.